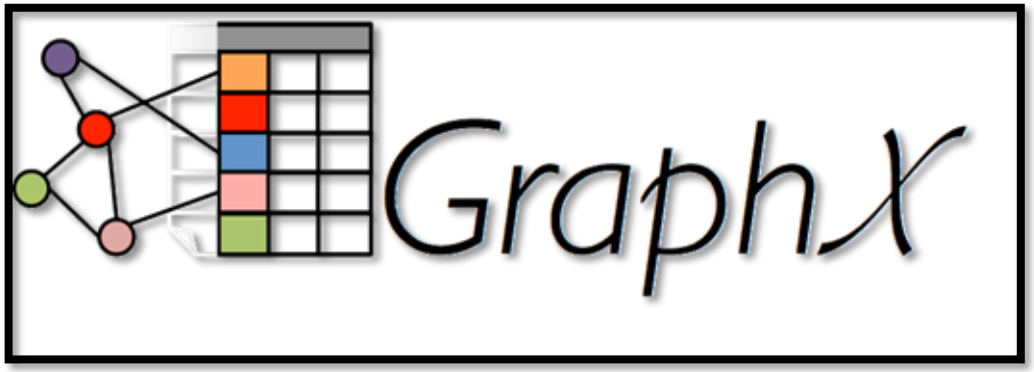
Big Graph Analytics

UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

GraphX



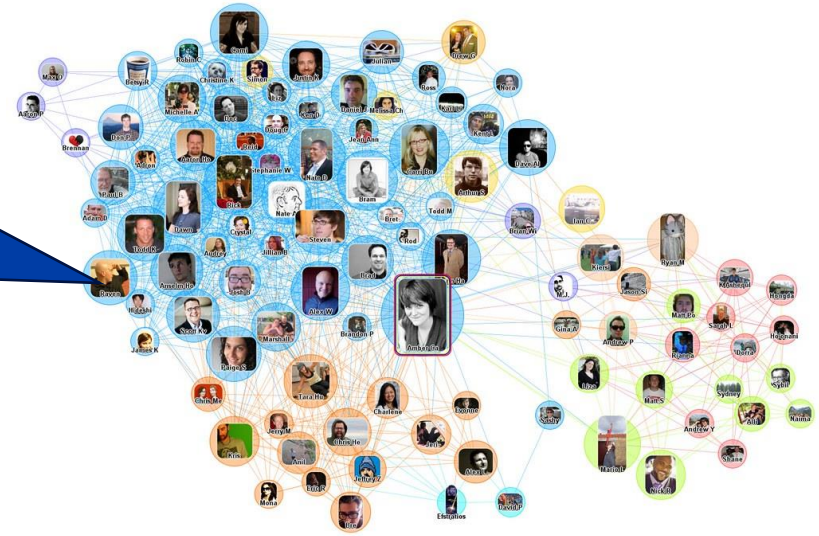
Spark Core (RDD)

HDFS

Big Graphs

- Graphs with hundreds of millions of vertices and billions of edges

1.7 billion users
100s of billions of connections

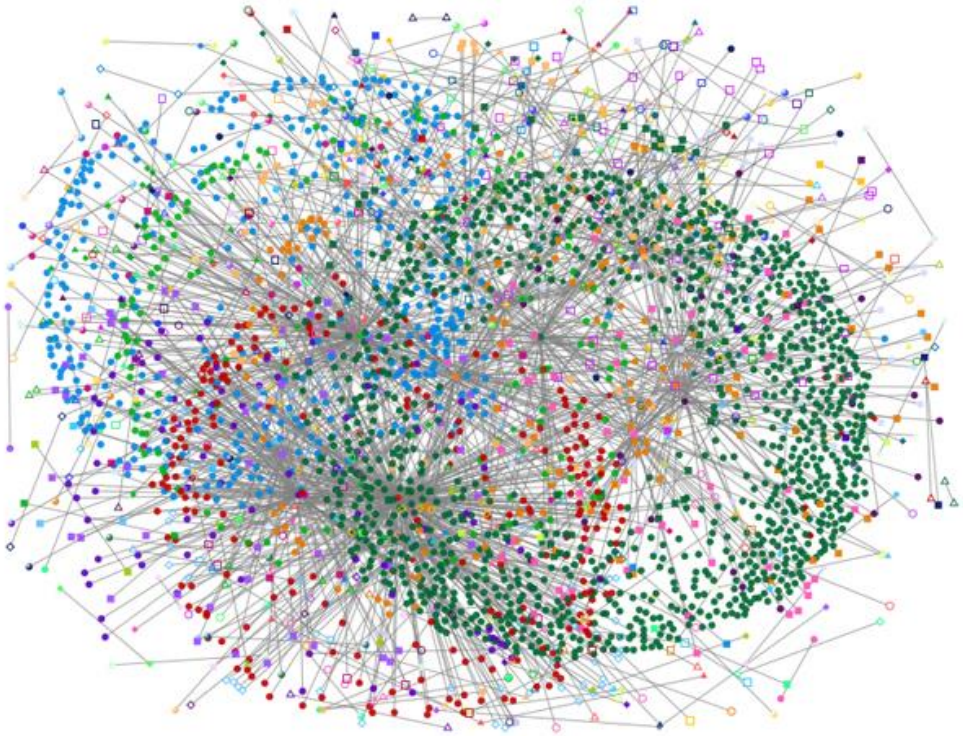


- The graph is too big to fit on one machine

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Web Graphs

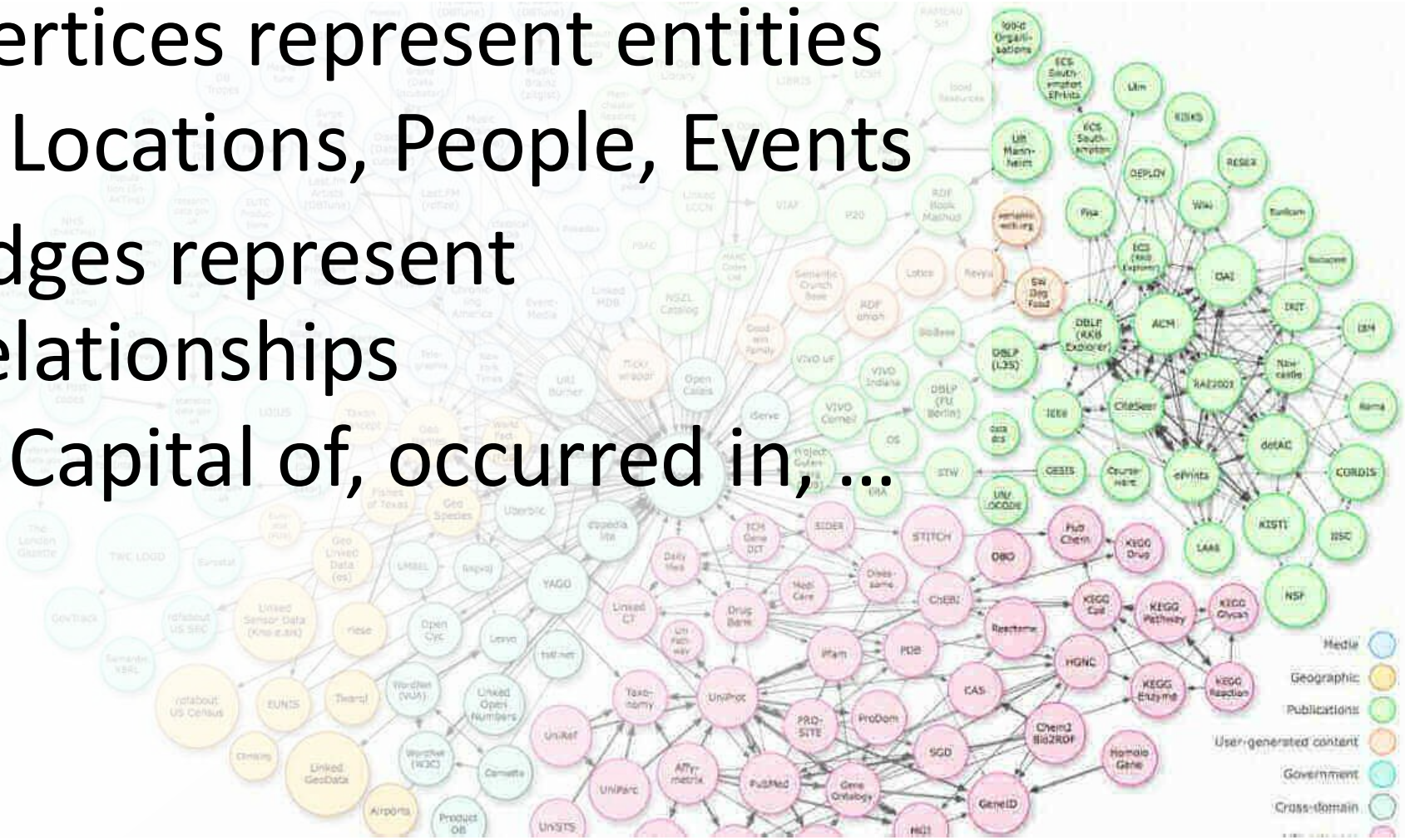
- Vertices represent pages
- Edges represent hyper links



UCR CELEBRATING 30 YEARS
Marlan and Rosemary Bourns
College of Engineering

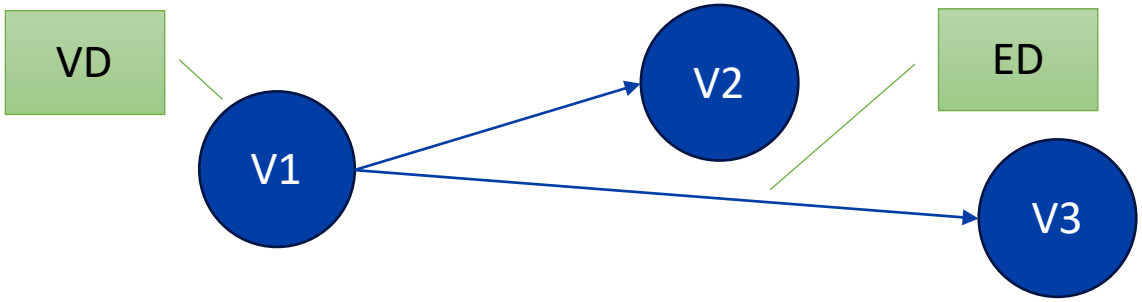
Knowledge Graphs

- Vertices represent entities
  - Locations, People, Events
- Edges represent relationships
  - Capital of, occurred in, …



UCR
CELEBRATING 30 YEARS
Marlan and Rosemary Bourns College of Engineering

Property Graph Model



Graph[VD, ED]

VertexRDD[VD]

<VertexID, VD>
<VertexID, VD>
…

EdgeRDD[ED]

<VertexID, VertexID, ED>
<VertexID, VertexID, ED>
….

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Examples of Graphs

- Knowledge Graph
  - Graph[String, String]
  - VD: String – entity names
  - ED: String – relationship
- Road Network
  - VD: (longitude, Latitude)
  - ED: (StreetName, SpeedLimit, …)

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Graph Creation

- Direct creation from RDDs

```
val vertices: RDD[(VertexId, VD)]
val edges: RDD[Edge[ED]]
val defaultVertex: VD
val graph = Graph(vertices, edges, defaultVertex)
```

- Loading from an edge list text file

```
val graph = GraphLoader.
  edgeListFile(sc, ”edges.txt")
```

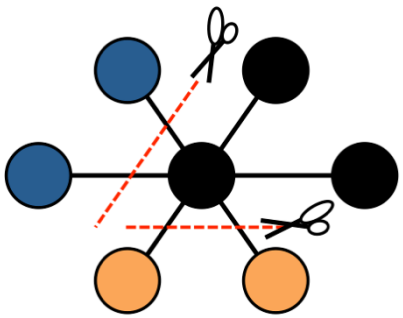
Edges.txt

```
# This is a comment
2 1
4 1
1 2
```

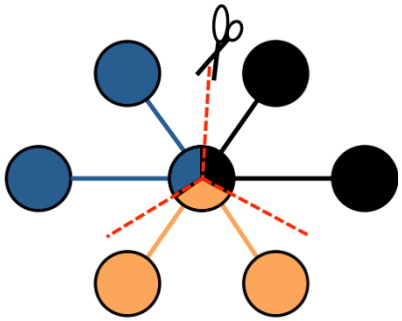

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Graph Partitioning

- Edge cut
  - Each vertex is assigned to one partition
  - Some edges are replicated
- Vertex cut
  - Each edge is assigned one partition
  - Some vertices are replicated
- GraphX adopts Vertex Cut to minimize communication cost



Edge Cut



Vertex Cut

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Simple Graph Processing

- Vertices and edges can be treated as RDDs
- Regular RDD operations can operate on them

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Vertex Operations

- Filter: Keeps only a subset of vertices that satisfy the predicate
- mapValues: Modify the value on each vertex
- diff(other: VertexRDD[VD]): Removes all vertices in the other set of vertices
- For more details, see:
  - https://spark.apache.org/docs/latest/graphx-programming-guide.html#vertexrdds

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

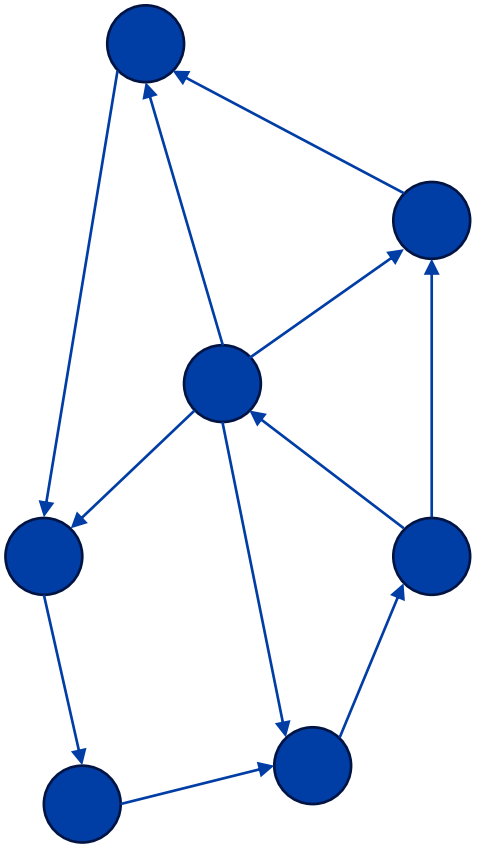
Edge Operations

- mapValues: Modify the value on each edge
- Reverse: Reverse the direction of each edge (V1 -> V2) → (V2 -> V1)
- For more details, see:
  - https://spark.apache.org/docs/latest/graphx-programming-guide.html#edgerdds

CELEBRATING 30 YEARS
Marlan and Rosemary Bourns College of Engineering

Iterative Graph Processing

- Each vertex send a message along outgoing edges
- Each vertex updates its vertex data by aggregating incoming messages
- Repeat until a stopping condition



UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Pregel API

```
def pregel[A](initialMsg: A, maxIterations:
Int, activeDirection: EdgeDirection)(
    vprog: (VertexId, VD, A) => VD,
    sendMsg: EdgeTriplet[VD, ED] =>
Iterator[(VertexId, A)],
    mergeMsg: (A, A) => A)
  : Graph[VD, ED]
```

https://spark.apache.org/docs/latest/graphx-programming-guide.html#pregel-api

EdgeDirection
{In, Out, Both, Either}

EdgeTriplet[VD, ED]
Source Attribute: VD
Destination Attribute: VD
Edge Attribute: ED

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Graph Analytics: Connected Components

- Goal: Assign each vertex the smallest ID in its connected component.
- Message: Each vertex sends its current label (initially its own ID) to neighbors.
- Update Rule: On each superstep, a vertex adopts the smallest label received from its neighbors if it's smaller than its current label.
- Termination: When no vertex changes its label (i.e., messages stop propagating), the algorithm converges.

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Graph Analytics: Single-source Shortest Path (SSSP)

- Goal: Find the minimum distance from a source vertex to all others.
- Message: Each vertex sends updated distance estimates to neighbors (e.g., its own distance + edge weight).
- Update Rule: A vertex keeps the smallest distance received and forwards new distance estimates if updated.
- Termination: When no shorter distances are propagated, the algorithm halts.

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Graph Analytics: Page Rank

- Goal: Compute an "importance" score for each vertex based on its incoming links.
- Message: Each vertex sends a fraction of its current rank (e.g., rank / out-degree) to neighbors.
- Update Rule: Each vertex sums incoming contributions, applies damping (e.g., new_rank = 0.15 + 0.85 * sum), and updates its rank.
- Termination: After a fixed number of iterations or when rank changes fall below a threshold.

UCR CELEBRATING 30 YEARS Marlan and Rosemary Bourns College of Engineering

Further Reading

- Official Programming Guide
  - https://spark.apache.org/docs/latest/graphx-programming-guide.html
- GraphX paper:
  - Xin, Reynold S., Joseph E. Gonzalez, Michael J. Franklin, and Ion Stoica. "GraphX: A resilient distributed graph system on spark." In First international workshop on graph data management experiences and systems, pp. 1-6. 2013.